LAPORAN PRAKTIKUM
STRUKTUR DATA

MODUL 5
SINGLY LINKED LIST (BAGIAN 2)



Disusun Oleh :

NAMA : Fauziah Istiqomah

NIM : 109102530004

Dosen:

Diah Septiana, S.Kom., M.C.S.(AI).

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2026

A. Teori Singkat

Singly linked list adalah struktur data berupa rangkaian elemen yang saling berkait lewat pointer dan bersifat fleksibel, karena dapat tumbuh dan mengerut sesuai kebutuhan. Berbeda dengan array yang ukurannya statis, linked list diimplementasikan dengan pointer yang bersifat dinamis: memori dialokasikan saat dibutuhkan dan dibebaskan saat tidak dipakai. Disebut singly karena setiap elemen hanya memiliki satu arah pointer, yaitu next yang menunjuk ke elemen berikutnya, sehingga penelusuran data hanya bisa dilakukan maju dari elemen pertama (First) sampai batas akhir (Nil).

Komponen utama pada elemen singly linked list terdiri dari:

1. **info:** komponen penyimpan nilai atau data.
2. **next:** pointer yang menunjuk ke elemen selanjutnya di dalam list.

Karena singly linked list tidak memiliki indeks, akses data ke posisi tertentu tidak dapat dilakukan secara langsung. Oleh karena itu, pencarian data (searching) dilakukan dengan menelusuri pointer next secara sekuensial (satu per satu) dari elemen pertama hingga data ditemukan atau penelusuran mencapai Nil.

Searching adalah operasi dasar untuk memeriksa keberadaan suatu data di dalam list. Penelusuran dimulai dari elemen pertama (First), lalu setiap elemen dikunjungi satu per satu melalui pointer next. Pada setiap elemen, info dibandingkan dengan nilai yang dicari. Jika sama, operasi mengembalikan alamat (address) elemen tersebut, dan jika penelusuran mencapai Nil tanpa menemukan nilai itu, operasi mengembalikan Nil yang berarti data tidak ada. Selain untuk menemukan data, searching juga menjadi prasyarat sebelum operasi lanjutan seperti insertAfter, delAfter, atau delP, karena operasi tersebut membutuhkan alamat elemen yang dituju.

Operasi pada singly linked list dikelompokkan sebagai berikut:

- **Pencarian elemen** (findElm): menelusuri list dari First sampai elemen yang dicari ditemukan atau mencapai Nil.
- **Penambahan elemen:** insertFirst (di awal list), insertAfter (setelah elemen Prec), dan insertLast (di akhir list).
- **Penghapusan elemen:** delFirst, delLast, delAfter, dan delP. Elemen yang dihapus dikembalikan lewat parameter alamat, lalu di-dealokasi.
- **Proses terhadap seluruh elemen:** printInfo (menampilkan semua info), nbList (menghitung jumlah elemen), delAll (menghapus semua elemen), invertList (membalik urutan elemen), serta copyList dan fCopyList (menyalin list).

Untuk menjaga keteraturan struktur program, implementasi singly linked list pada C++ umumnya dibagi menjadi tiga file, yaitu file deklarasi (.h), file implementasi (.cpp), dan file program utama (main.cpp).

B. Latihan 5.2

Soal Nomor 2

Buatlah ADT *Singly Linked list* sebagai berikut di dalam *file* **"Singlylist.h"**:

```
Type infotype : int
Type address  : pointer to ElmList

Type ElmList <
   info : infotype
   next : address
>

Type List : < First : address >

procedure CreateList( input/output L : List )
function alokasi( x : infotype ) → address
procedure dealokasi(input/output P : address )
procedure printInfo( input L : List )
procedure insertFirst(input/output L : List, input P :address)
```

Kode (Singlylist.h)

```
#ifndef SINGLYLIST_H_INCLUDED
#define SINGLYLIST_H_INCLUDED
```

```
#include <iostream>
using namespace std;

#define Nil NULL
#define info(P) (P)->info
#define next(P) (P)->next
#define First(L) ((L).First)

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

// Deklarasi fungsi dasar
void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertFirst(List &L, address P);

// Deklarasi fungsi pencarian dan penjumlahan
address findElm(List L, infotype x);
int sumInfo(List L);

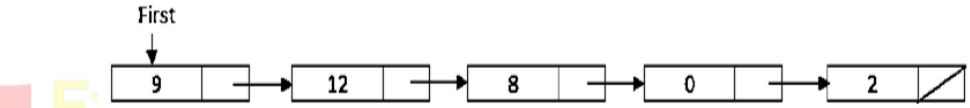
#endif
```

Deskripsi:

File Singlylist.h berfungsi sebagai deklarasi kerangka struktur data. File ini mendefinisikan tipe data bentukan (*struct*) untuk elemen list (ElmList) yang memuat komponen info dan next, serta mendefinisikan tipe data List yang

memuat pointer First. Selain itu, file ini berisi seluruh prototipe fungsi dan prosedur operasional, meliputi pembuatan list, alokasi dan dealokasi memori, penyisipan elemen, operasi pencarian (*findElm*), dan operasi agregasi data (*sumInfo*).

Kemudian buat implementasi ADT *Singly Linked list* pada *file* **"Singlylist.cpp".**
Adapun isi data



Gambar 5-1 Ilustrasi elemen

Kode (Singlylist.cpp)

```
#include "Singlylist.h"

void createList(List &L) {
    First(L) = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != Nil) {
        info(P) = x;
        next(P) = Nil;
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void printInfo(List L) {
    address P = First(L);
```

```
    while (P != Nil) {
        cout << info(P) << " ";
        P = next(P);
    }
    cout << endl;
}

void insertFirst(List &L, address P) {
    next(P) = First(L);
    First(L) = P;
}

address findElm(List L, infotype x) {
    address P = First(L);
    while (P != Nil) {
        if (info(P) == x) {
            return P;
        }
        P = next(P);
    }
    return Nil;
}

int sumInfo(List L) {
    int total = 0;
    address P = First(L);
    while (P != Nil) {
        total += info(P);
        P = next(P);
    }
    return total;
}
```

Deskripsi:

File Singlylist.cpp merupakan implementasi operasional dari seluruh prototipe yang dideklarasikan pada file header. File ini memuat algoritma secara rinci, meliputi instruksi inisialisasi list kosong, proses manajemen

alokasi pointer di dalam memori, logika pemindahan pointer pada metode penyisipan elemen di awal list (insertFirst), proses iterasi untuk pencarian nilai elemen secara sekuensial, serta algoritma perulangan untuk mengakumulasikan seluruh nilai elemen di dalam list.

Cobalah hasil implementasi ADT pada *file* "**main.cpp**"

```
int main()
{
    List L;
    address P1, P2, P3, P4, P5 = Nil;
    createList(L);

    P1 = alokasi(2);
    insertFirst(L,P1);

    P2 = alokasi(0);
    insertFirst(L,P2);

    P3 = alokasi(8);
    insertFirst(L,P3);

    P4 = alokasi(12);
    insertFirst(L,P4);

    P5 = alokasi(9);
    insertFirst(L,P5);

    printInfo(L)
    return 0;
}
```

```
9 12 8 0 2

Process returned 0 (0x0)   execution time : 0.019 s
Press any key to continue.
```

Gambar 5-2 *Output* Singlylist

Kode (main_soal2.cpp)

```
#include "Singlylist.h"

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2); insertFirst(L, P1);
```

```
    P2 = alokasi(0); insertFirst(L, P2);
    P3 = alokasi(8); insertFirst(L, P3);
    P4 = alokasi(12); insertFirst(L, P4);
    P5 = alokasi(9); insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

Output

```
PS C:\Users\istiq\OneDrive\STRUKTUR DATA\LAPRAK MODUL 5> .\soal2
9 12 8 0 2
```

Deskripsi:

Program diawali dengan membuat list kosong menggunakan createList. Kemudian lima elemen berisi nilai 2, 0, 8, 12, dan 9 dialokasikan satu per satu dengan alokasi, dan setiap elemen langsung disisipkan ke list menggunakan insertFirst. Karena insertFirst selalu menyisipkan elemen baru di awal list, elemen yang dimasukkan paling akhir (9) menjadi elemen pertama, sehingga urutan elemen di dalam list terbalik dari urutan penyisipannya. Setelah semua elemen dimasukkan, printInfo menelusuri list dari First sampai Nil dan mencetak info setiap elemen, sehingga output yang tampil adalah 9 12 8 0 2.

Soal Nomor 3

Carilah elemen dengan info 8 dengan membuat fungsi baru.
`fungsi findElm( L : List, x : infotype ) : address`

```
8 ditemukan dalam list
Process returned 0 (0x0)   execution time : 0.020 s
Press any key to continue.
```

Gambar 5-3 *Output* pencarian 8

Kode (main_soal3.cpp)

```cpp
#include "Singlylist.h"

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2); insertFirst(L, P1);
    P2 = alokasi(0); insertFirst(L, P2);
    P3 = alokasi(8); insertFirst(L, P3);
    P4 = alokasi(12); insertFirst(L, P4);
    P5 = alokasi(9); insertFirst(L, P5);

    address found = findElm(L, 8);
    if (found != Nil) {
        cout << info(found) << " ditemukan dalam list" << endl;
    } else {
        cout << "Elemen tidak ditemukan" << endl;
    }

    return 0;
}
```

Output

```
PS C:\Users\istiq\OneDrive\STRUKTUR DATA\LAPRAK MODUL 5> .\soal3
8 ditemukan dalam list
```

Deskripsi:

List dibuat dan diisi dengan cara yang sama seperti pada soal nomor 2. Setelah itu fungsi findElm(L, 8) dipanggil untuk mencari elemen yang memiliki info bernilai 8. Fungsi ini menelusuri list dari elemen pertama secara sekuensial; jika info suatu elemen sama dengan 8, fungsi mengembalikan alamat elemen tersebut, dan jika penelusuran mencapai Nil

tanpa hasil, fungsi mengembalikan Nil. Hasil pencarian disimpan pada pointer found. Jika found tidak sama dengan Nil, program mencetak info elemen tersebut diikuti kalimat "ditemukan dalam list", sedangkan jika sama dengan Nil program mencetak "Elemen tidak ditemukan". Karena nilai 8 ada di dalam list, output yang tampil adalah "8 ditemukan dalam list".

Soal Nomor 4

Hitunglah jumlah total info seluruh elemen (9+12+8+0+2=31).

```
Total info dari kelima elemen adalah 31
Process returned 0 (0x0)   execution time : 0.019 s
Press any key to continue.
```

Gambar 5-4 *Output* total info elemen

Kode (main_soal4.cpp)

```cpp
#include "Singlylist.h"

int main() {
    List L;
    address P1, P2, P3, P4, P5;

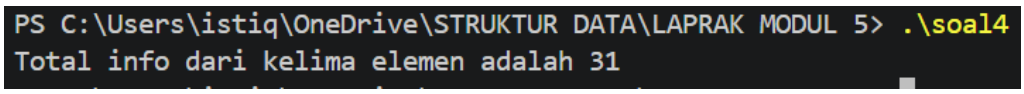
    createList(L);

    P1 = alokasi(2); insertFirst(L, P1);
    P2 = alokasi(0); insertFirst(L, P2);
    P3 = alokasi(8); insertFirst(L, P3);
    P4 = alokasi(12); insertFirst(L, P4);
    P5 = alokasi(9); insertFirst(L, P5);

    int total = sumInfo(L);
    cout << "Total info dari kelima elemen adalah " << total <<
endl;

    return 0;
}
```

Output



Deskripsi:

List dibuat dan diisi dengan cara yang sama seperti pada soal nomor 2 dan 3. Kemudian fungsi sumInfo(L) dipanggil untuk menjumlahkan seluruh info elemen di dalam list. Di dalam fungsi tersebut, variabel total diinisialisasi dengan 0, lalu list ditelusuri dari elemen pertama sampai Nil dan info setiap elemen ditambahkan ke total. Perhitungannya adalah 9 + 12 + 8 + 0 + 2 = 31. Nilai yang dikembalikan disimpan pada variabel total dan dicetak, sehingga output yang tampil adalah “Total info dari kelima elemen adalah 31”.